Claude API, Claude Code e Agent SDK: quale usare e quando

> Appunti per la certificazione CCA-F (tutti i domini). Spiegano la differenza tra i tre modi di costruire con Claude, cosa ti dà ciascuno "gratis" e cosa devi fare tu, e come riconoscere nelle domande d'esame quale strumento è in gioco.
>
> ⚠️ Nomi di pacchetti, flag e opzioni possono cambiare tra versioni. Per i dettagli aggiornati fa fede la documentazione ufficiale: **docs.claude.com** → **API**, **Claude Code** e **Agent SDK**.

Indice

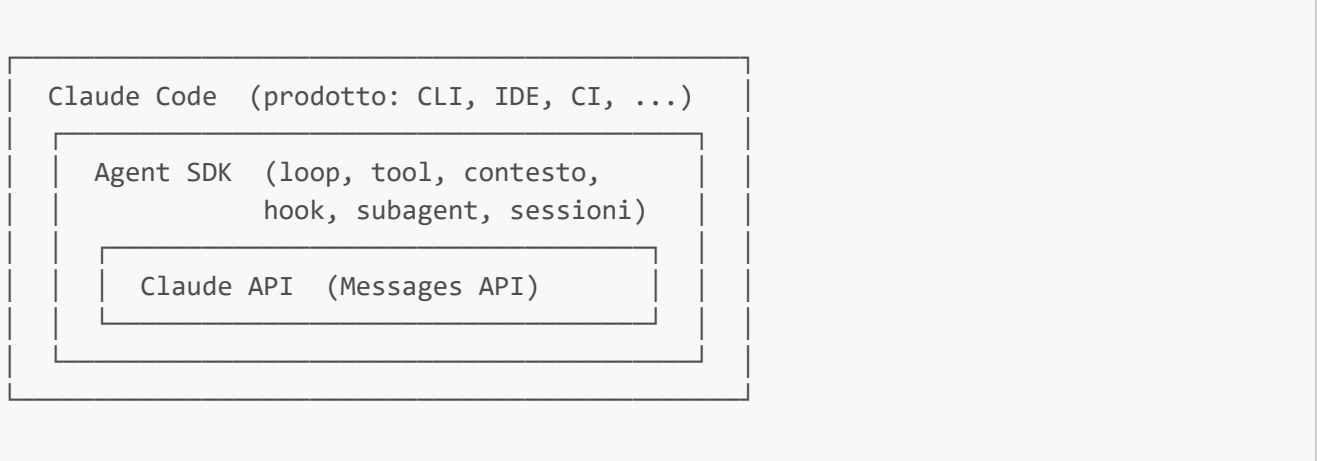
1. L'idea in una riga
2. Claude API
3. Claude Code
4. Agent SDK
5. Il loop agentico: chi lo esegue?
6. Tabella di confronto
7. Concetti condivisi e concetti specifici
8. Come scegliere
9. Gli scenari d'esame: quale strumento c'è dietro
10. Trappole e domande da esame

---

1. L'idea in una riga

Strumento	In una riga
Claude API	Il **motore**: mandi messaggi, ricevi una risposta. Tutto il resto lo costruisci tu.
Claude Code	Un **agente pronto all'uso** per lo sviluppo software: terminale, IDE, desktop, web, CI.
Agent SDK	Lo **stesso motore agentico di Claude Code, come libreria**, per costruire i *tuoi* agenti.

Un'immagine utile:



Ogni livello **aggiunge** qualcosa a quello sotto. Più sali, più cose sono già pronte; più scendi, più controllo hai (e più lavoro).

---

2. Claude API

Cos'è

L'accesso **diretto** ai modelli tramite HTTP (o gli SDK client in Python, TypeScript, ecc.). L'endpoint principale è la **Messages API**: invii `system`, `messages`, eventualmente `tools`, e ricevi una risposta.

Cosa ti dà

- **Messages API**: conversazioni, system prompt, output testuale.
- **Tool use**: definisci gli strumenti con `name`, `description`, `input_schema` (JSON Schema). Il modello **chiede** di usarli; **sei tu a eseguirli**.
- `tool_choice`: `auto` (decide il modello), `any` (deve usare un tool qualsiasi), `{"type": "tool", "name": ...}` (deve usare quello), `none`.
- `stop_reason`: `tool_use` (vuole un tool), `end_turn` (ha finito), `max_tokens` (troncato), ecc.
- **Message Batches API**: richieste asincrone, circa **50% di costo**, completamento **entro 24 ore**, risultati correlati da `custom_id`, **nessun loop di tool** dentro una singola richiesta.
- **Prompt caching**: riuso di un **prefisso statico** (system prompt, documenti) per ridurre costo e latenza.
- **Streaming**, conteggio dei token, input multimodale (immagini, PDF).

Cosa NON ti dà

- Nessun loop agentico automatico: **lo scrivi tu**.
- Nessuno strumento integrato per file, shell, ricerca nel codice.
- Nessuna gestione automatica del contesto, delle sessioni, dei permessi.
- **Stateless**: a ogni chiamata devi rimandare tutta la cronologia che serve.

Quando usarla

- Compiti **a singolo turno** o con flusso fisso: classificazione, estrazione, riassunto.
- **Pipeline di estrazione strutturata** (tool use come "schema forzato").
- Elaborazioni **massive non urgenti** → Batches.
- Quando vuoi **controllo totale** su ogni chiamata, ogni token, ogni passaggio.

---

3. Claude Code

Cos'è

Un **agente di programmazione** già pronto, che lavora nel tuo repository. Si usa da **terminale (CLI)**, **IDE** (VS Code, JetBrains), **app desktop**, **web**, e in modalità **non interattiva** nelle pipeline CI.

Cosa ti dà già pronto

- **Strumenti integrati**: Read, Write, Edit, Bash, Grep, Glob, WebFetch, WebSearch, ...
- **Loop agentico** completo, gestione del contesto, `/compact`, `/clear`.

- **Memoria di progetto**: `CLAUDE.md` (utente, progetto, sottocartella), `@import`, `.claude/rules/` con `paths`.
- **Estensioni**: comandi `/` (`.claude/commands/`), **skill** (`.claude/skills/`), **subagent**, **hook** (`PreToolUse`, `PostToolUse`, ...), **server MCP** (`.mcp.json`, `~/.claude.json`).
- **Permessi**: regole allow/deny in `settings.json`, `settings.local.json`, impostazioni gestite dall'organizzazione.
- **Modalità di lavoro**: plan mode, esecuzione diretta, subagent *Explore*.
- **Sessioni**: `--continue`, `--resume`, fork.

Modalità non interattiva (CI)

```
claude -p "Rivedi questa PR per problemi di sicurezza" \
  --output-format json \
  --json-schema schema.json
```

- **`-p` / `--print`**: esegue, stampa il risultato ed esce. Senza `-p` in CI il job **resta bloccato** in attesa di input.
- **`--output-format json`** e **`--json-schema`**: output strutturato e validato per gli script a valle.
- Anche in `-p` Claude Code **legge il `CLAUDE.md` del progetto**: le convenzioni non vanno duplicate nel prompt della CI.
- Limita gli strumenti al minimo necessario (es. `Bash(npm test:*)`, solo lettura per la review).

Quando usarlo

- Lavoro **sul codice**: generazione, refactoring, debugging, documentazione, review.
- Automazione del ciclo di sviluppo in **CI/CD**.
- Quando ti serve un agente **subito**, configurabile con file, senza scrivere il loop.

---

4. Agent SDK

Cos'è

La libreria (**Claude Agent SDK**, in Python e TypeScript; in passato chiamata *Claude Code SDK*) che espone **la stessa infrastruttura agentica di Claude Code** — loop, strumenti, gestione del contesto, hook, subagent, sessioni, MCP — per costruire **agenti propri**, anche **non legati al codice** (supporto clienti, ricerca, operazioni).

Cosa ti dà

- Il **loop agentico** già implementato: il modello chiede un tool, l'SDK lo esegue, rimanda il risultato, continua fino alla fine.
- Gli **strumenti integrati** (Read, Write, Bash, Grep, Glob, WebSearch, ...) **e** i tuoi strumenti personalizzati, spesso esposti come **server MCP** (anche in-process).
- **`allowedTools` / `disallowedTools`**: quali strumenti l'agente può usare.
- **Subagent** definiti in `agents`, lanciati con lo strumento **`Task`** (che deve essere in `allowedTools`).
- **Hook** programmatici (`PreToolUse`, `PostToolUse`, ...) per regole **deterministiche**: bloccare un rimborso sopra soglia, normalizzare le date nei risultati.
- **Sessioni**: `resume`, `fork_session`.

- **Permessi** e modalità di permesso.
- Può leggere la configurazione in stile Claude Code (`CLAUDE.md`, skill, comandi) se lo abiliti.

```
options = ClaudeAgentOptions(
    system_prompt="Sei un agente di supporto clienti...",
    allowed_tools=["mcp__support__get_customer",
                   "mcp__support__lookup_order",
                   "mcp__support__process_refund",
                   "mcp__support__escalate_to_human"],
    hooks={"PreToolUse": [refund_limit_hook]},   # regola deterministica
)
```

Quando usarlo

- Vuoi un **agente autonomo** nel tuo prodotto o servizio, con **strumenti tuoi**.
- Ti serve la robustezza di Claude Code (loop, contesto, hook, subagent) **senza** riscriverla.
- Sistemi **multi-agente** con coordinatore e subagent.

---

5. Il loop agentico: chi lo esegue?

È la differenza pratica più importante.

Con la Claude API: lo scrivi tu

```
messages = [{"role": "user", "content": richiesta}]
while True:
    resp = client.messages.create(model=..., tools=tools, messages=messages)
    messages.append({"role": "assistant", "content": resp.content})
    if resp.stop_reason != "tool_use":       # "end_turn" → fine
        break
    results = []
    for block in resp.content:
        if block.type == "tool_use":
            try:
                out = esegui(block.name, block.input)
                results.append({"type": "tool_result",
                                "tool_use_id": block.id, "content": out})
            except Exception as e:
                results.append({"type": "tool_result", "tool_use_id": block.id,
                                "content": str(e), "is_error": True})
    messages.append({"role": "user", "content": results})
```

Responsabilità tue:

- decidere quando fermarti **in base a** `stop_reason` (mai facendo il parsing del testo, tipo "ho finito");
- eseguire gli strumenti e rimandare i risultati;

- restituire gli errori come `tool_result` con `is_error: true` e un messaggio utile (mai una stringa vuota: il modello penserebbe che sia andato tutto bene);
- gestire la cronologia e la dimensione del contesto.

> 💡 Gli SDK client offrono anche aiuti (es. un *tool runner*) che automatizzano parte di questo ciclo, ma il concetto da esame resta: **con l'API il loop è responsabilità tua**.

Con Claude Code e Agent SDK: è già fatto

Il loop, l'esecuzione degli strumenti, la gestione degli errori dei tool e del contesto sono **integrati**. Tu configuri: prompt, strumenti, permessi, hook, subagent.

---

6. Tabella di confronto

	Claude API	**Claude Code**	**Agent SDK**
Tipo	API / libreria client	Prodotto (CLI, IDE, desktop, web)	Libreria per costruire agenti
Per chi	Chi integra Claude in un'applicazione	Sviluppatori, pipeline CI	Chi costruisce agenti propri
Loop agentico	**Lo scrivi tu**	Integrato	Integrato
Strumenti integrati	No (solo tool che definisci tu)	Sì (Read, Edit, Bash, Grep, ...)	Sì, più i tuoi
Strumenti personalizzati	Tool definiti con JSON Schema	Server MCP	Server MCP / tool personalizzati
Configurazione	Parametri della richiesta	File: `CLAUDE.md`, `.claude/`, `settings.json`, `.mcp.json`	Codice (`options`) + opzionalmente file
Hook	No (logica nel tuo codice)	Sì, in `settings.json`	Sì, come funzioni nel codice
Subagent	Da costruire a mano	Sì	Sì (`agents` + `Task`)
Sessioni	Stateless	`--continue`, `--resume`, fork	`resume`, `fork_session`
Batches / caching	Sì, direttamente	Non è il suo scopo	Si appoggia all'API
Ambito tipico	Estrazione, classificazione, chat, batch	Sviluppo software, review, CI	Supporto clienti, ricerca, strumenti per sviluppatori

---

7. Concetti condivisi e concetti specifici

Validi ovunque (sono proprietà del modello)

- Prompt con **criteri espliciti** invece di istruzioni vaghe.
- **Few-shot** per formati e casi ambigui.

- **Descrizioni dei tool** chiare e distintive.
- **Tool use / schema** elimina errori di **sintassi**, non di **semantica**.
- Gestione del contesto: tagliare gli output, "lost in the middle", blocchi di fatti persistenti.

Specifici della Claude API

- `tool_choice`, `stop_reason`, `is_error` nel `tool_result` che costruisci tu.
- **Message Batches API**, `custom_id`.
- **Prompt caching** (prefisso statico prima del contenuto variabile).

Specifici di Claude Code

- `CLAUDE.md` (gerarchia, `@import`, `/memory`), `.claude/rules/` con `paths`.
- Comandi `/`, skill (`SKILL.md`, `context: fork`, `allowed-tools`), plan mode.
- `-p`, `--output-format json`, `--json-schema` per la CI.
- `.mcp.json` (progetto) vs `~/.claude.json` (utente).

Specifici dell'Agent SDK (ma con gli stessi concetti di Claude Code)

- `allowedTools`, `agents`, strumento `Task`.
- Hook come funzioni (`PreToolUse` per bloccare, `PostToolUse` per normalizzare).
- `resume`, `fork_session`.

---

8. Come scegliere

```
Il compito è sviluppo software nel tuo repository (o CI)?
 └─ Sì → Claude Code
 └─ No → Serve un agente autonomo che usa strumenti in più passaggi?
         └─ Sì → Agent SDK
         └─ No → Claude API
                 └─ Tante richieste non urgenti? → Message Batches API
```

Esigenza	Scelta
Revisione automatica delle PR in GitHub Actions	Claude Code con `-p`
Agente di supporto clienti con `get_customer`, `process_refund`	Agent SDK
Estrarre campi da 250.000 documenti nel weekend	Claude API + Message Batches
Classificare un'email in tempo reale	Claude API (sincrona)
Sistema di ricerca con coordinatore e subagent	Agent SDK
Aiutare un team a fare refactoring con regole condivise	Claude Code + `CLAUDE.md`
Controllo totale su ogni chiamata e ogni token	Claude API

---

9. Gli scenari d'esame: quale strumento c'è dietro

Scenario	Strumento	Concetti tipici
Customer Support Resolution Agent	**Agent SDK** + MCP	hook, escalation, errori dei tool, case facts
Code Generation with Claude Code	**Claude Code**	`CLAUDE.md`, rules, comandi, skill, plan mode
Multi-Agent Research System	**Agent SDK**	`Task`, `agents`, contesto isolato, provenienza
Developer Productivity with Claude	**Agent SDK** (strumenti integrati) + MCP	Grep/Glob/Read/Edit, MCP, contesto
Claude Code for Continuous Integration	**Claude Code** non interattivo	`-p`, `--json-schema`, review indipendente
Structured Data Extraction	**Claude API**	tool use, `tool_choice`, schema, Batches, retry con errori

Leggere la prima riga dello scenario ti dice già **quali meccanismi esistono** e quindi quali opzioni sono plausibili.

---

10. Trappole e domande da esame

Opzione che sembra giusta	Perché è sbagliata
"Con l'API il modello esegue da solo lo strumento"	Con l'API il modello **chiede**, sei tu che esegui e rimandi il risultato
"Fermati quando il testo contiene 'ho finito'"	Il loop si governa con `stop_reason`
"In caso di eccezione del tool, restituisci una stringa vuota"	Il modello crede che sia riuscito: usa `is_error: true` con il messaggio
"Lancia `claude \"...\"` nel job di CI"	Senza `-p` resta interattivo e si blocca
"In CI Claude Code non legge `CLAUDE.md`"	Lo legge anche in `-p`
"Usa la Batches API per l'agente che chiama strumenti in più turni"	Una richiesta batch non esegue un loop di tool
"Usa la Batches API per la review che blocca il merge"	Fino a 24h, nessuna garanzia di latenza
"Il coordinatore ha i subagent in `agents` ma non `Task` negli strumenti"	Senza `Task` non può delegare
"Imposta `tool_choice` forzato per avere valori corretti"	Garantisce la **chiamata**, non la **correttezza** dei valori
"Riscrivi da zero il loop di Claude Code con l'API per avere un agente di supporto"	L'Agent SDK lo fornisce già

Opzione che sembra giusta	Perché è sbagliata
"Usa Claude Code per estrarre dati da milioni di documenti"	È uno strumento di sviluppo: per l'estrazione massiva usa l'API (e Batches)
"L'API ricorda la conversazione precedente"	È stateless: la cronologia va rimandata

Parole chiave → strumento

Se leggi...	Pensa a...
`stop_reason`, `tool_choice`, `tool_result`, `custom_id`	Claude API
Message Batches, 24 ore, 50%	Claude API (Batches)
`CLAUDE.md`, `.claude/rules`, `/comando`, plan mode, `/compact`	Claude Code
`-p`, `--output-format json`, `--json-schema`, GitHub Actions	Claude Code in CI
`allowedTools`, `agents`, `Task`, `fork_session`	Agent SDK
hook `PreToolUse` / `PostToolUse`	Claude Code **o** Agent SDK
`.mcp.json`, server MCP	Claude Code **o** Agent SDK

> **Mantra:** l'**API** è il motore, **Claude Code** è l'agente pronto per il codice, l'**Agent SDK** è il motore agentico per costruire il tuo agente. Più sali, meno costruisci; più scendi, più controlli.